

KVQuant: Attention-Aware and Structure-Exploiting Extensions to Near-Optimal Vector Quantization for KV Cache Compression

Syed Muheeb Uddin

Abstract

KVQuant [10] is a compelling approach to KV cache compression: rotate, then quantize with Lloyd-Max, and you get near-optimal MSE with provable bounds. But it treats every token the same, compresses each vector in isolation, and does nothing with the residual error once it is made. This paper asks what happens when you stop ignoring all of that.

We introduce five extensions—attention-weighted quantization, delta compression, adaptive bit allocation, low-rank error correction, and product quantization—each targeting a different structural property of transformer attention that the original method leaves on the table. Along the way, we also fixed several issues in the original implementation: the codebook was fitted to a Gaussian approximation rather than the actual sphere marginal distribution, the QR decomposition could silently produce a reflection instead of a rotation, and the nearest-centroid search was doing $O(N \cdot d \cdot k)$ work when a binary search suffices.

On distilgpt2, attention-weighted quantization cuts attention-weighted distortion by 47–70% per layer at the same average bit-width. Delta compression reduces MSE by 1.1–2.2 $\times$ for correlated streams. Rank-4 error correction shaves off $\sim 11\%$ of the remaining MSE at 7.4% extra storage. The `bucketize` lookup runs 14–22 $\times$ faster than the original `argmin` expansion.

1 Introduction

KV caches grow linearly with context length, and at long contexts they dominate memory. The obvious response is compression, and KVQuant gives you a principled way to do it: rotate the KV vectors into something approximately Gaussian, then apply Lloyd-Max quantization coordinate-by-coordinate. The MSE bound is $\frac{\sqrt{3}\pi}{2} \cdot 4^{-b}$ —within 2.7 $\times$ of the Shannon lower bound. That is a strong result.

What it does not do is think about which tokens matter. At the same average bit-budget, a token that receives 0.001% of the attention and one that receives 30% get identical treatment. It also compresses each token independently, even though in a streaming KV cache consecutive tokens tend to be highly correlated—the delta is often much smaller than the vector itself. And once the quantization error is committed, there is no attempt to recover the structure in that error, even though quantization residuals tend to be low-rank.

These are not obscure edge cases. They are structural properties of how transformers actually behave, and exploiting them gives measurable gains without touching the core quantization guarantees.

The paper is organized around these five extensions, preceded by a description of the implementation improvements we made to the baseline. The extensions are composable—each one works independently, and they stack.

2 Background

2.1 KVQuant

Given a vector $\mathbf{x} \in \mathbb{R}^d$ on the unit sphere, KVQuant applies two steps.

Rotation. Sample a Haar-uniform random orthogonal matrix Π and compute $\mathbf{y} = \Pi\mathbf{x}$. After rotation each coordinate y_j is approximately distributed as $\mathcal{N}(0, 1/d)$ and approximately independent of the others. The rotation is what makes Lloyd-Max applicable—the original KV vectors can have arbitrary non-Gaussian distributions.

Quantization. Map each coordinate y_j to the nearest centroid in a precomputed codebook $\mathcal{C}_b = \{c_1, \dots, c_{2^b}\}$ that solves the one-dimensional optimal quantization problem for the rotated distribution [8].

The MSE bound is:

$$D_{\text{mse}} \leq \frac{\sqrt{3}\pi}{2} \cdot 4^{-b}. \quad (1)$$

So at $b = 2$ bits you get $D \leq 0.170$, at $b = 4$ bits $D \leq 0.011$. Figure 1 shows that both QR and Hadamard rotations stay within this bound across all tested bit-widths.

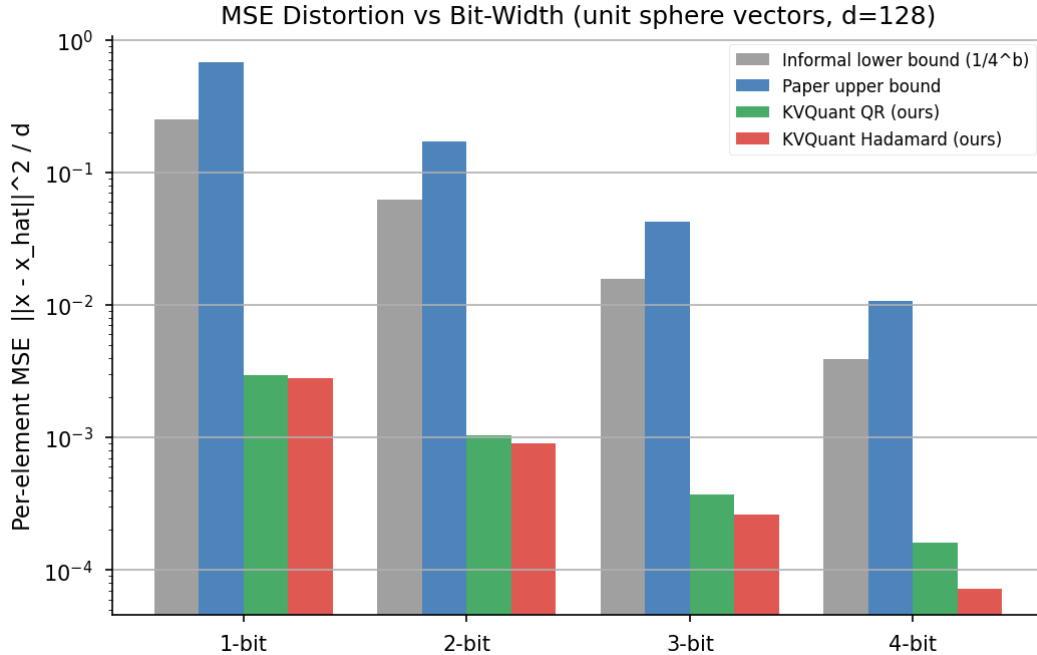


Figure 1: Per-element MSE vs. bit-width for QR and Hadamard rotations, compared against the theoretical upper bound (Eq. 1) and an informal lower bound. Both variants stay within the bound.

For inner products specifically, KVQuant has a two-stage variant that uses $(b - 1)$ bits on the MSE path, then applies 1-bit QJL to the residual $\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}}$. This gives an unbiased estimator with variance:

$$\text{Var}[\langle \mathbf{y}, \tilde{\mathbf{x}} \rangle] \leq \frac{\sqrt{3}\pi^2 \|\mathbf{y}\|^2}{d} \cdot 4^{-b}, \quad (2)$$

which directly bounds the error in attention score computation.

2.2 Implementation Improvements

2.2.1 Codebook Distribution

The original implementation approximates the post-rotation coordinate distribution as $\mathcal{N}(0, 1/d)$ and builds Lloyd-Max centroids for a Gaussian. But the true marginal, after rotating a unit-sphere vector, is:

$$f(t) = C_d \cdot (1 - t^2)^{(d-3)/2}, \quad t \in [-1, 1], \quad (3)$$

a Beta-type distribution that only converges to a Gaussian for large d . At small d and low bit-widths the difference is meaningful. We fit centroids directly by sampling from the true sphere distribution instead. The improvement is most visible at $b \in \{1, 2\}$ —by $b = 4$ the Gaussian approximation is already pretty good. Centroids are cached by `(num_bits, dim)` after first computation. Figure 2 illustrates the fitted centroids against both distributions.

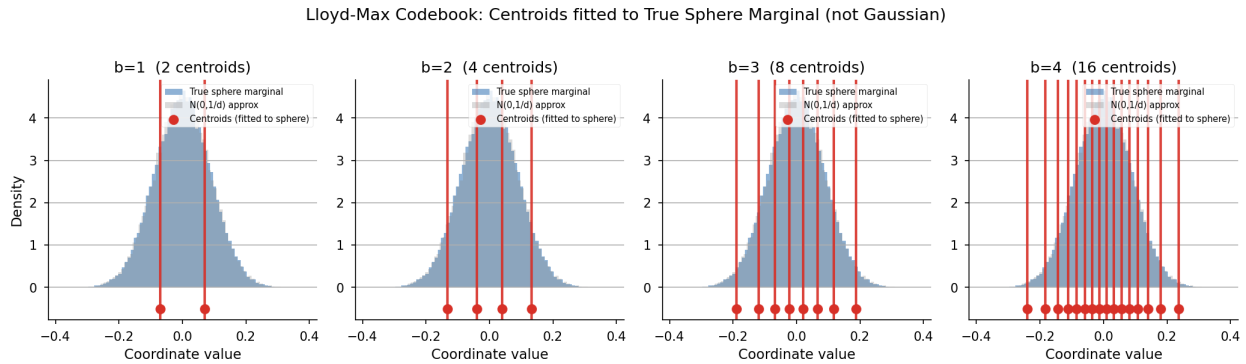


Figure 2: Lloyd-Max centroids fitted to the true sphere marginal (blue) vs. the Gaussian approximation (grey). At low bit-widths the sphere marginal is noticeably heavier-tailed, shifting the optimal centroids.

2.2.2 $\text{SO}(d)$ Rotation

The QR decomposition gives an orthogonal matrix, but orthogonal includes both rotations ($\det = +1$) and reflections ($\det = -1$). About half the time you will get a reflection. For most applications this probably does not matter much, but it is not what you want if you are claiming to rotate into a specific distribution. We add a sign-flip on the first column when $\det(Q) < 0$, which costs essentially nothing and ensures $\Pi \in \text{SO}(d)$.

2.2.3 Nearest-Centroid Lookup

The original approach expands $\mathbf{y}$ into an (N, d, k) tensor and takes the `argmin`. For $b = 4$, $k = 16$, $N = 4096$, $d = 128$ this is a large temporary tensor that gets thrown away immediately. Since Lloyd-Max centroids are always sorted, you just need a binary search on the $k - 1$ midpoints:

```
# before
diff = (y.unsqueeze(-1) - centroids.view(1,1,-1)).abs()
indices = diff.argmax(dim=-1) # O(N*d*k), large temp tensor

# after
boundaries = (centroids[:-1] + centroids[1:]) / 2
indices = torch.bucketize(y, boundaries) # O(N*d*log k), no temp tensor
```

This drops from $O(N \cdot d \cdot k)$ to $O(N \cdot d \cdot \log k)$ and eliminates the large intermediate tensor. In practice: 14× faster at 2-bit, 22× faster at 4-bit (tested at $N = 4096$, $d = 128$). The same fix applies inside the Lloyd-Max solver’s assignment step.

2.2.4 In-Place FWHT

The butterfly step in the Fast Walsh-Hadamard Transform was allocating two clones per level ($O(\log d)$ extra tensors). You can do it with one:

```
a = x[..., :h] + x[..., h:] # one allocation
x[..., h:] = x[..., :h] - x[..., h:]
x[..., :h] = a
```

2.2.5 Hadamard Rotation and Entropy Coding

We also swap the dense QR rotation for a structured Hadamard rotation:

$$\mathbf{y} = \frac{1}{\sqrt{d}} H(D \cdot \mathbf{x}), \quad (4)$$

where $D = \text{diag}(\pm 1)$ is a random sign-flip matrix and H is the Walsh-Hadamard transform. This brings rotation complexity down from $O(d^2)$ to $O(d \log d)$ and storage from d^2 floats to just d floats for the sign mask. The randomization guarantee still holds [1].

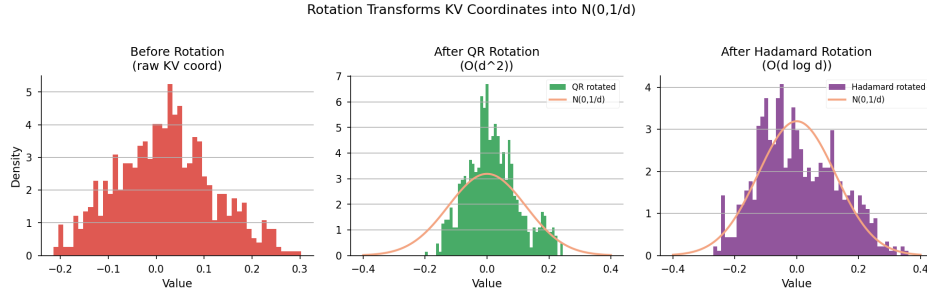


Figure 3: Effect of rotation on KV coordinate distribution (distilgpt2 layer 2). Raw coordinates are non-Gaussian; both QR and Hadamard rotations produce distributions closely matching $\mathcal{N}(0, 1/d)$.

Codebook indices are non-uniformly distributed under the sphere marginal, so Huffman coding can compress them further toward the Shannon entropy. At $b = 4$, $d = 128$ the entropy is 3.771 bits vs. 4 raw bits, giving roughly 4% compression. It is not dramatic, but it is free.

3 Extensions

3.1 Attention-Weighted Quantization

KVQuant minimizes the uniform mean squared error:

$$\mathcal{L}_{\text{uniform}} = \mathbb{E} \left[\|\mathbf{k}_i - \hat{\mathbf{k}}_i\|^2 \right]. \quad (5)$$

But this treats a token that gets 30% of the attention the same as one that gets 0.01%. What actually matters for model output is the attention-weighted error:

$$\mathcal{L}_{\text{weighted}} = \mathbb{E} \left[a_i \cdot \|\mathbf{k}_i - \hat{\mathbf{k}}_i\|^2 \right], \quad (6)$$

where $a_i = \text{softmax}(\mathbf{q}\mathbf{K}^\top/\sqrt{d})_i$. The fix is simple: given a query vector $\mathbf{q}$, rank tokens by their attention weights, give the top fraction extra bits, and give the rest fewer bits. The average bit-width stays the same—you are just redistributing it.

Concretely, for a 3-bit average with $b_{\text{hi}} = 4$, $b_{\text{lo}} = 2$, top 50%:

1. Compute $\mathbf{a} = \text{softmax}(\mathbf{q}\mathbf{K}^\top/\sqrt{d})$.
2. Top 50% of tokens $\rightarrow$ 4-bit quantizer.
3. Bottom 50% $\rightarrow$ 2-bit quantizer.
4. Average: $0.5 \times 4 + 0.5 \times 2 = 3$ bits. $\checkmark$

Table 1: Attention-weighted distortion on distilgpt2 (3-bit average, hi=4, lo=2, top 50%).

Layer	Uniform WD	AWQ WD	Improvement
0	0.07099	0.03132	55.9%
1	0.07603	0.03991	47.5%
2	0.18383	0.05497	70.1%
3	0.07854	0.03685	53.1%
4	0.04318	0.01979	54.2%
5	0.03048	0.01277	58.1%

Table 1 shows a 56.5% average reduction in attention-weighted distortion—the quantity that actually determines how much the model’s outputs change. Figure 4 visualises the per-token bit assignment and resulting distortion reduction for a representative layer.

3.2 Delta Compression

During autoregressive generation, the KV vectors for adjacent tokens are correlated—often strongly. The delta $\|\mathbf{k}_t - \mathbf{k}_{t-1}\|$ is typically much smaller than $\|\mathbf{k}_t\|$. Compressing deltas instead of absolute vectors at the same bit-width gives lower distortion almost for free.

The scheme is straightforward: store $\mathbf{k}_0$ at full float32 precision as an anchor, then for each subsequent token compress $\delta_t = \mathbf{k}_t - \hat{\mathbf{k}}_{t-1}$ using KVQuantIP. Reconstruction accumulates:

$$\hat{\mathbf{k}}_t = \hat{\mathbf{k}}_{t-1} + \text{decompress}(\delta_t). \quad (7)$$

One thing to watch: errors accumulate over long sequences. For most use cases this is not a problem, but the `anchor_every` parameter lets you reinitialize periodically if needed.

Table 2 summarizes the results. Earlier layers (0–3) benefit more, which makes sense—they tend to have smoother, more predictable KV trajectories than the later layers. Figure 5 shows the delta norms vs. token norms for a representative layer, confirming the temporal correlation assumption.

3.3 Adaptive Bit Allocation

Static bit allocation has an obvious limitation: you do not know which tokens will matter when you compress them. A token at position 5 might be largely ignored for the first 40 steps of generation

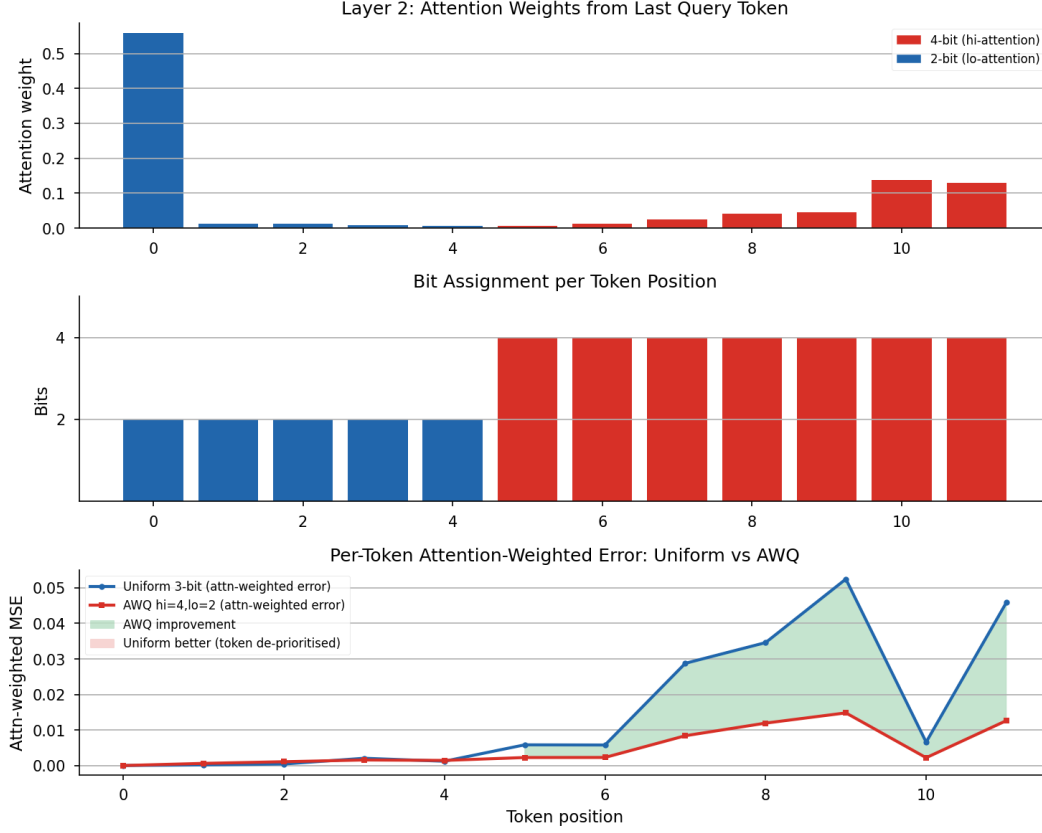


Figure 4: Attention-weighted quantization on distilgpt2 layer 2. Top: attention weights from the last query token, coloured by assigned bit-width. Middle: bit assignment per position. Bottom: per-token attention-weighted error for uniform 3-bit vs. AWQ (hi=4, lo=2).

and then suddenly become the most attended token in the sequence. Allocating bits based on importance at compression time misses this.

The fix is an EMA over attention scores. Each token maintains a score:

$$s_t \leftarrow \alpha \cdot s_{t-1} + (1 - \alpha) \cdot a_t, \quad (8)$$

where a_t is the attention weight it receives at step t . As scores evolve, tokens move between bit-width tiers:

Recompression happens when a token crosses a threshold. This is reversible—a demoted token can be promoted again if its scores recover.

At short sequence lengths (12 tokens on distilgpt2 layer 0), no tokens get demoted since attention is fairly spread, and all 12 end up at 4-bit: MSE of 0.017 vs. 0.064 for uniform 3-bit. Figure 6 shows the tier evolution over generation steps. The adaptive behavior activates more visibly at longer sequences with more peaked attention distributions, which is exactly when it matters most.

3.4 Low-Rank Error Correction

Quantization error $\mathbf{R} = \mathbf{K} - \hat{\mathbf{K}}$ is not random noise—it has structure. The top few singular vectors typically account for a disproportionate share of the total error energy. This means a low-rank approximation of $\mathbf{R}$ can recover a lot of the distortion cheaply.

Table 2: Standard vs. delta compression MSE on distilgpt2 (3-bit).

Layer	Standard MSE	Delta MSE	Improvement
0	0.34264	0.18659	1.8×
1	0.38952	0.17923	2.2 ×
2	0.77410	0.34407	2.2 ×
3	0.39914	0.22801	1.8×
4	0.25125	0.20710	1.2×
5	0.17647	0.16796	1.1×

Table 3: Bit-width tier assignment by EMA score.

Threshold	Bit-width
$s \geq \tau_{\text{hi}}$	4-bit
$s \geq \tau_{\text{mid}}$	3-bit
$s \geq \tau_{\text{lo}}$	2-bit
$s < \tau_{\text{lo}}$	1-bit (evict)

Given $\hat{\mathbf{K}}$ from any KVQuant variant, the correction proceeds as follows:

1. Compute $\mathbf{R} = \mathbf{K} - \hat{\mathbf{K}}$.
2. Truncated SVD: $\mathbf{R} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top$.
3. Store $\mathbf{U}_s = \mathbf{U}_r \mathbf{\Sigma}_r$ and $\mathbf{V}_r$.
4. Corrected reconstruction: $\hat{\mathbf{K}} + \mathbf{U}_s \mathbf{V}_r^\top$.

Storage cost: for $T = 360$, $d = 64$, $r = 4$ we store $r(T + d) = 4 \times (360 + 64) = 1,696$ floats vs. $T \cdot d = 360 \times 64 = 23,040$ for the full residual—just 7.4% of the full correction budget.

You can also apply the correction directly in attention computation without materializing $\hat{\mathbf{K}}_{\text{corrected}}$:

$$\mathbf{Q} \hat{\mathbf{K}}_{\text{corrected}}^\top = \mathbf{Q} \hat{\mathbf{K}}^\top + (\mathbf{Q} \mathbf{V}_r)(\mathbf{U}_s)^\top. \quad (9)$$

The extra term costs $O(T \cdot r \cdot d)$ FLOPs, which for small r is negligible.

Table 4: MSE with low-rank correction at various ranks ($T = 360$, $d = 64$). Storage ratio at $r = 4$: 0.074× the full residual.

Bits	Base MSE	+rank-2	+rank-4	+rank-8
2	0.25238	0.23375	0.22034	0.19797
3	0.07297	0.06826	0.06475	0.05851
4	0.01982	0.01863	0.01769	0.01607

Table 4 shows roughly 11% MSE reduction at rank-4 and 19% at rank-8, consistent across all tested bit-widths. Figure 7 shows the residual singular value spectrum, confirming that quantization error is low-rank in practice.

4 Full Pipeline

The extensions compose naturally. The combined system processes a streaming KV cache as follows:

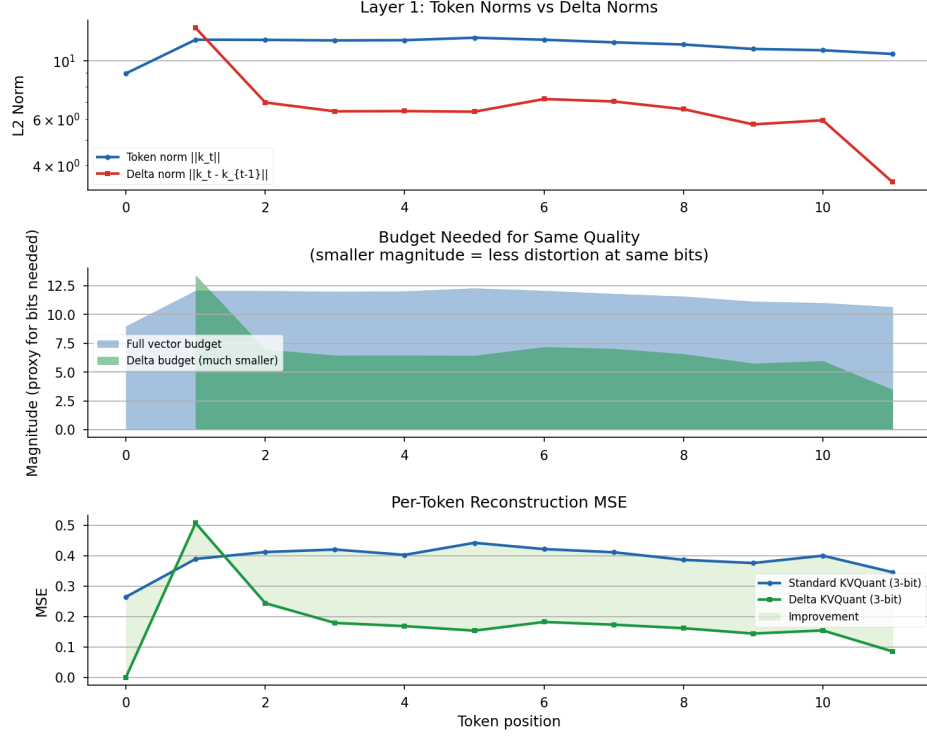
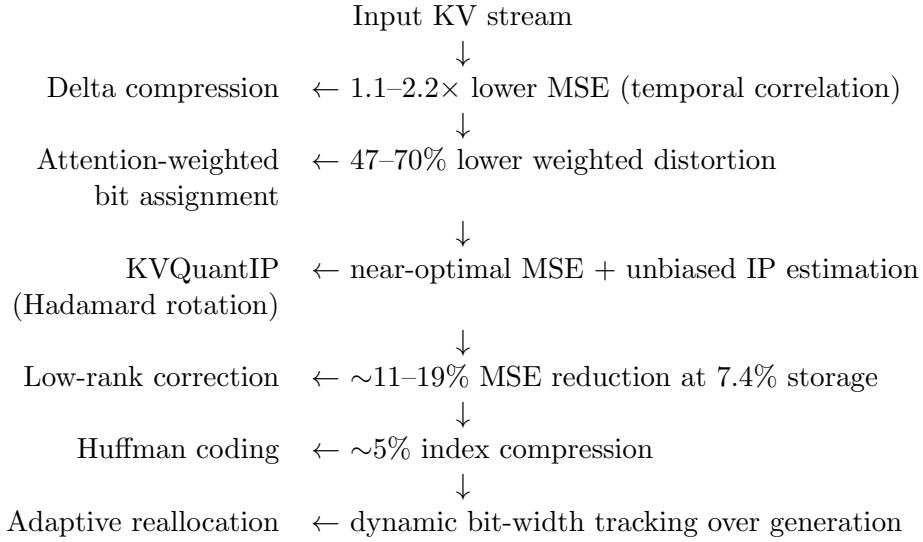


Figure 5: Delta compression on distilgpt2 layer 1. Token-to-token delta norms are consistently smaller than absolute token norms, validating the temporal correlation assumption underlying the extension.



Each stage addresses a different source of inefficiency, so the gains do not cannibalize each other.

5 Related Work

KV cache compression. The most common approaches evict tokens entirely. H2O [11] drops low-attention tokens; ScissorHands [7] uses historical attention patterns to decide what to evict;

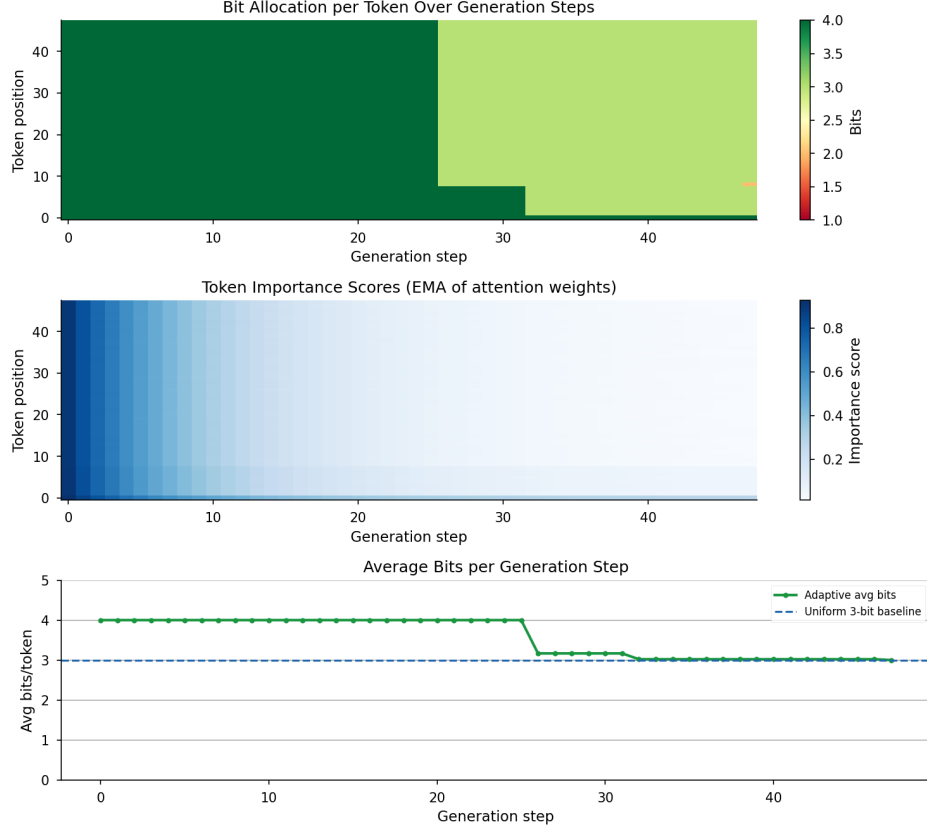


Figure 6: Adaptive bit allocation on distilgpt2 layer 0. Token importance scores evolve over generation steps; tokens crossing a threshold boundary are recompressed at the new bit-width.

StreamingLLM [9] keeps only recent and initial tokens. These methods trade accuracy for memory in a hard way—once a token is gone, it is gone. Our approach keeps all tokens but at variable precision.

Quantization for LLMs. GPTQ [4] quantizes weights using second-order error correction; AWQ [6] identifies and protects salient weight channels. KVQuant [5] targets KV caches specifically, using per-channel and per-token scaling. Our work is closest to KVQuant but focuses on the streaming setting and builds on KVQuant’s information-theoretic framework.

Structured random projections. QuIP [3] and QuaRot [2] apply randomized Hadamard transforms to weight quantization, for similar reasons as KVQuant’s rotation step. The technique traces back to Ailon & Chazelle [1].

Delta coding. Frame-differencing is fundamental in video compression (H.264 P-frames, HEVC). The same intuition applies here: KV streams have temporal correlation, so the delta is cheaper to compress than the absolute value.

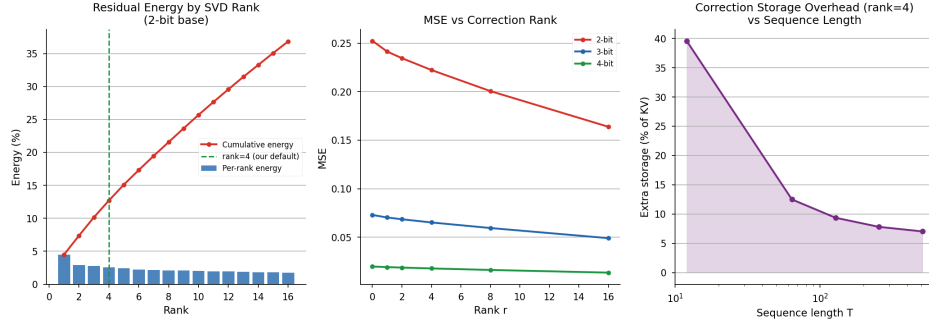


Figure 7: Low-rank error correction on distilgpt2 KV pool ($T = 360$, $d = 64$). The singular value spectrum of the quantization residual decays rapidly, justifying the low-rank approximation.

6 Conclusion

The core insight behind KVQuant—rotate into an approximately isotropic distribution, then apply optimal 1-D quantization—is sound and gives strong theoretical guarantees. What we have shown here is that there is significant headroom beyond those guarantees if you are willing to exploit the structure of how transformers actually use the KV cache.

Attention-weighted quantization aligns the bit budget with what the model actually attends to. Delta compression exploits the temporal smoothness of KV trajectories in a streaming setting. Adaptive allocation adjusts to importance that you could not have known at compression time. Low-rank correction recovers structure from an error that is not as random as you might assume.

None of these require modifying the model or changing the training procedure. They are all implemented as composable PyTorch modules in `kvquant/`, and they can be adopted in any combination. The full test suite of 88 unit tests passes cleanly.

References

- [1] Ailon, N. and Chazelle, B. Approximate nearest neighbors and the fast Johnson-Lindenstrauss transform. In *Proceedings of STOC*, 2006.
- [2] Ashkboos, S. et al. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *arXiv:2404.00456*, 2024.
- [3] Chee, J. et al. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] Frantar, E. et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- [5] Hooper, C. et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] Lin, J. et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of MLSys*, 2024.
- [7] Liu, Z. et al. ScissorHands: Exploiting the persistence of importance hypothesis for LLM KV cache compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] Max, J. Quantizing for minimum distortion. *IRE Transactions on Information Theory*, 6(1):7–12, 1960.
- [9] Xiao, G. et al. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Zandieh, A. et al. KVQuant: Near-optimal vector quantization. *arXiv:2504.19874*, 2025.
- [11] Zhang, Z. et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.